

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса

Группы ИИ-26

Петручик Д.А.

Проверила:

Андренко К.В.

Цель работы: Научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Ход работы:

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
10	STL-10 (размеченная часть)	96X96	Adam

Код программы:

```
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import
DataLoader
from PIL import Image
```

```
transform = transforms.Compose(
    [transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5,
    0.5), (0.5, 0.5, 0.5))]
)
```

```
custom_transform =
transforms.Compose(
    [transforms.Resize((96, 96)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5,
    0.5), (0.5, 0.5, 0.5))]
```

)

```
train_dataset =  
torchvision.datasets.STL10(  
    root="./data", split="train",  
    download=True,  
    transform=transform  
)
```

```
train_loader = DataLoader(  
    dataset=train_dataset,  
    batch_size=64,  
    shuffle=True  
)
```

```
test_dataset =  
torchvision.datasets.STL10(  
    root="./data", split="test",  
    download=True,  
    transform=transform  
)
```

```
test_loader = DataLoader(  
    dataset=test_dataset,  
    batch_size=64,  
    shuffle=False  
)
```

```
class CNN(nn.Module):  
    def __init__(self):  
        super(CNN, self).__init__()  
  
        self.conv1 = nn.Conv2d(  
            in_channels=3,  
            out_channels=16,  
            kernel_size=3,  
            padding=1  
        )  
  
        self.pool = nn.MaxPool2d(  
            kernel_size=2,  
            stride=2  
        )  
  
        self.conv2 = nn.Conv2d(  
            in_channels=16,  
            out_channels=32,  
            kernel_size=3,  
            padding=1  
        )  
  
        self.fc1 = nn.Linear(
```

```

        32 * 24 * 24,
        128
    )

    self.fc2 = nn.Linear(
        128,
        10
    )

    def forward(self, x):

        x = self.pool(
            torch.relu(
                self.conv1(x)
            )
        )

        x = self.pool(
            torch.relu(
                self.conv2(x)
            )
        )

        x = x.view(
            -1,
            32 * 24 * 24
        )

        x = torch.relu(
            self.fc1(x)
        )

        x = self.fc2(x)

        return x

```

```

model = CNN()

```

```

criterion = nn.CrossEntropyLoss()

```

```

optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)

```

```

epoch_losses = []

```

```

epochs = 10

```

```

for epoch in range(epochs):

    running_loss = 0.0

    for i, data in
enumerate(train_loader, 0):

        inputs, labels = data

        optimizer.zero_grad()

        outputs = model(inputs)

        loss = criterion(
            outputs,
            labels
        )

        loss.backward()

        optimizer.step()

        running_loss += loss.item()

    avg_loss = running_loss /
len(train_loader)

    epoch_losses.append(avg_loss)

    print(
        f"Epoch {epoch+1} Loss:
{avg_loss:.4f}"
    )

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, epochs + 1),
    epoch_losses,
    marker="o",
    color="b",
    label="Train Loss"
)

plt.title(
    "График изменения ошибки
при обучении (Loss)"
)

plt.xlabel(
    "Эпоха"
)

```

```

plt.ylabel(
    "Loss (CrossEntropy)"
)

plt.grid(True)

plt.xticks(
    range(1, epochs + 1)
)

plt.legend()

plt.show()

correct = 0

total = 0

model.eval()

with torch.no_grad():
    for data in test_loader:
        images, labels = data

        outputs = model(images)

        _, predicted = torch.max(
            outputs.data,
            1
        )

        total += labels.size(0)

        correct += (
            predicted == labels
        ).sum().item()

accuracy = 100 * correct / total

print(
    f"\nТочность сети на {total}
    тестовых изображениях:
    {accuracy:.2f}%"
)

sota_results = {

    "ResNet-50": 86.4,

```

```

    "WideResNet-28-10": 87.0,

    "EfficientNet-B7": 87.7,

    "SimCLR": 91.8,

    "SwAV": 92.1

}

models = list(sota_results.keys())
+ ["Моя CNN"]

accuracies =
list(sota_results.values()) +
[accuracy]

plt.figure(figsize=(10, 6))

bars = plt.bar(
    models,
    accuracies
)

plt.title(
    "Сравнение точности модели
с SOTA на STL-10"
)

plt.ylabel(
    "Accuracy (%)"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.ylim(
    0,
    100
)

for bar, value in zip(bars,
accuracies):

```

```
plt.text(
    bar.get_x() +
    bar.get_width()/2,
    value + 1,
    f'{value:.1f}%',
    ha="center"
)
```

```
plt.grid(
    axis="y"
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("\nСравнение с SOTA:")
```

```
for name, acc in
sota_results.items():
```

```
    difference = accuracy - acc
```

```
    print(
        f'{name}: {acc:.1f}% |
Разница моей CNN:
{difference:+.2f}%'
    )
```

```
print(
    f'Моя CNN: {accuracy:.2f}%'
)
```

```
classes = train_dataset.classes
```

```
dataiter = iter(test_loader)
```

```
images, labels = next(dataiter)
```

```
images_to_show = images[:3]
```

```
labels_to_show = labels[:3]
```

```
custom_image_path =
"image_f2a326.jpg"
```

```
custom_img = Image.open(
    custom_image_path
).convert("RGB")
```

```
custom_tensor =
custom_transform(
    custom_img
).unsqueeze(0)
```

```
with torch.no_grad():
```

```
    outputs_test =
model(images_to_show)
```

```
    probs_test = F.softmax(
        outputs_test,
        dim=1
    )
```

```
    conf_test, predicted_test =
torch.max(
    probs_test,
    1
)
```

```
    output_custom =
model(custom_tensor)
```

```
    probs_custom = F.softmax(
        output_custom,
        dim=1
    )
```

```
    conf_custom, predicted_custom
= torch.max(
    probs_custom,
    1
)
```

```
fig, axes = plt.subplots(
    1,
    4,
    figsize=(16, 5)
)
```

```
for i in range(3):
```

```
    img_display =
```

```
images_to_show[i].numpy() * 0.5  
+ 0.5
```

```
img_display = np.transpose(  
    img_display,  
    (1, 2, 0)  
)
```

```
axes[i].imshow(  
    img_display  
)
```

```
real_label = classes[  
    labels_to_show[i]  
]
```

```
pred_label = classes[  
    predicted_test[i]  
]
```

```
confidence = conf_test[i].item()  
* 100
```

```
axes[i].set_title(  
    f"Real: {real_label}\nPred:  
{pred_label}\nConf:  
{confidence:.1f}%"  
)
```

```
axes[i].axis(  
    "off"  
)
```

```
img_custom_display =  
custom_tensor.squeeze(0).numpy(  
) * 0.5 + 0.5
```

```
img_custom_display =  
np.transpose(  
    img_custom_display,  
    (1, 2, 0)  
)
```

```
axes[3].imshow(  
    img_custom_display  
)
```

```
pred_custom_label = classes[
    predicted_custom.item()
]
```

```
confidence_custom =
conf_custom.item() * 100
```

```
axes[3].set_title(
    f"Real: monkey
(Custom)\nPred:
{pred_custom_label}\nConf:
{confidence_custom:.1f}%"
)
```

```
axes[3].axis(
    "off"
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
plt.tight_layout()
plt.show()
```

Результат тестирования:

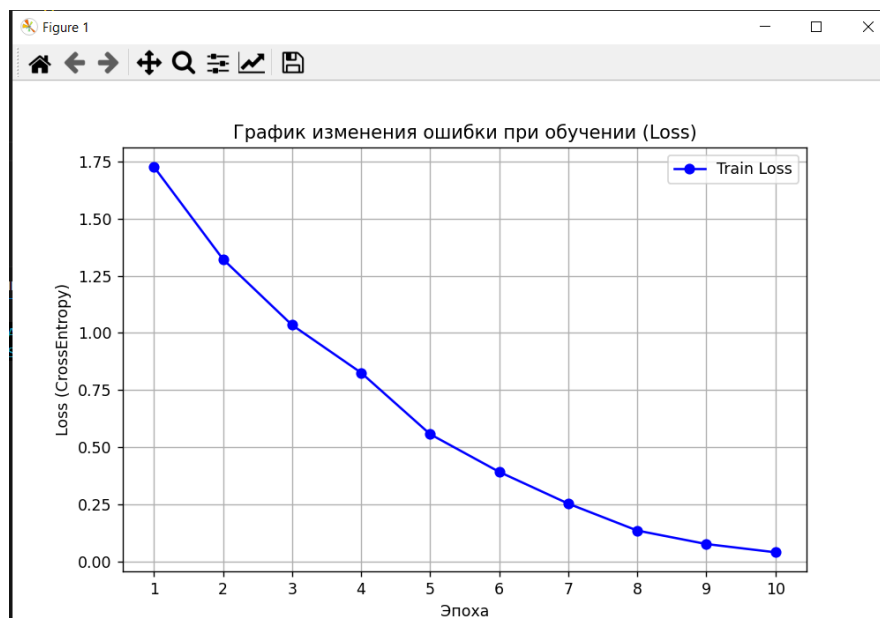
Epoch 1 Loss: 1.8596
Epoch 2 Loss: 1.3627
Epoch 3 Loss: 1.1272
Epoch 4 Loss: 0.9170
Epoch 5 Loss: 0.7252
Epoch 6 Loss: 0.5400
Epoch 7 Loss: 0.3940
Epoch 8 Loss: 0.2379
Epoch 9 Loss: 0.1537
Epoch 10 Loss: 0.0899

Точность сети на 8000 тестовых изображениях: 55.35%

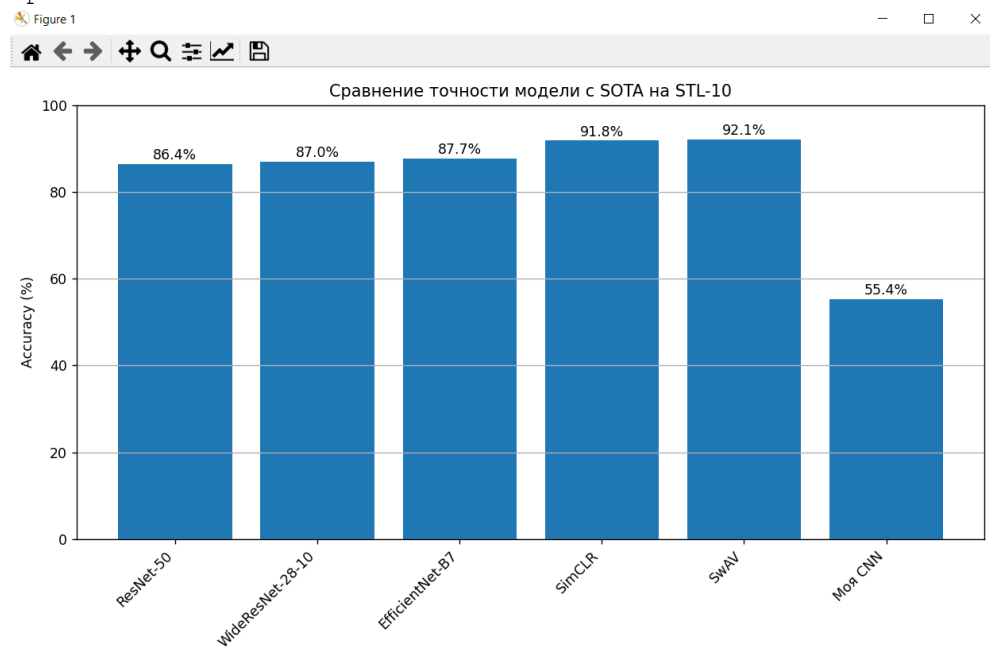
Сравнение с SOTA:

ResNet-50: 86.4% | Разница моей CNN: -31.05%
WideResNet-28-10: 87.0% | Разница моей CNN: -31.65%
EfficientNet-B7: 87.7% | Разница моей CNN: -32.35%
SimCLR: 91.8% | Разница моей CNN: -36.45%
SwAV: 92.1% | Разница моей CNN: -36.75%
Моя CNN: 55.35%

График изменения ошибки:



Сравнение с SOTA:



Точность сети на 8000 тестовых изображениях: 53.95%



Простая свёрточная сеть из базовых слоёв, обученная с помощью алгоритма Adam, реально способна выявлять паттерны и отличать объекты на цветных изображениях размером 96x96 из выборки STL-10. Тем не менее, у базовой архитектуры есть чёткие границы возможностей: из-за малого объема размеченной части STL-10 (всего 5000 обучающих примеров) модель быстро переобучается, путает визуально схожие классы животных или транспорта и теряется при подаче изображений, непохожих по ракурсу или стилю на обучающую выборку (что наглядно видно на примере сторонних картинок)

Вывод: Прodelав данную лабораторную работу я научился конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

